

Introduction

Stacking, also called stacked generalisation (Wolpert 1992), combines predictions from several diverse base learners using a second-level meta-learner trained to predict the target from the base-model predictions. Stacking consistently wins competitive ML benchmarks when base models are individually decent and sufficiently diverse — the meta-learner captures complementary strengths that no single model can match. It generalises bagging (one type of model averaged) and boosting (one type of model sequentially trained) to arbitrary heterogeneous ensembles.

Prerequisites

A working understanding of cross-validation, the bias-variance trade-off in ensembling, and the difference between in-sample and out-of-fold predictions.

Theory

The standard stacking workflow:

1. **Train base learners** $M_1, M_2, \dots, M_K$ on the training data; compute *out-of-fold* (OOF) predictions $\hat{y}_k^{\text{oof}}$ via cross-validation.
2. **Train a meta-learner** g (typically linear or regularised regression / logistic regression) to predict the target from the OOF predictions: $\hat{y} = g(\hat{y}_1^{\text{oof}}, \dots, \hat{y}_K^{\text{oof}})$.
3. **Refit base learners** on the full training set; produce final predictions as $\hat{y}(x) = g(M_1(x), \dots, M_K(x))$.

Using OOF predictions for the meta-learner prevents leakage that would occur if the meta-learner trained on in-sample base predictions.

Assumptions

Base learners are sufficiently diverse (a trees + linear + kNN combination usually outperforms five different tree variants); the meta-learner is simple enough that it does not over-fit the small set of base predictions.

R Implementation

```
library(tidymodels); library(stacks)

set.seed(2026)
n <- 500
df <- tibble(x1 = rnorm(n), x2 = rnorm(n), x3 = rnorm(n),
             y = factor(sample(c("pos", "neg"), n,
                              replace = TRUE, prob = c(0.3, 0.7))))

split <- initial_split(df, strata = y)
train <- training(split); test <- testing(split)

folds <- vfold_cv(train, v = 5)
ctrl <- control_stack_grid()

rec <- recipe(y ~ ., data = train)
rf_spec <- rand_forest(mtry = 2, trees = 300) %>%
  set_engine("ranger") %>% set_mode("classification")
log_spec <- logistic_reg(penalty = 0.01) %>%
  set_engine("glmnet") %>% set_mode("classification")

rf_wf <- workflow() %>% add_recipe(rec) %>% add_model(rf_spec)
```

```
log_wf <- workflow() %>% add_recipe(rec) %>% add_model(log_spec)

rf_res <- fit_resamples(rf_wf, folds, control = ctrl)
log_res <- fit_resamples(log_wf, folds, control = ctrl)

st <- stacks() %>% add_candidates(rf_res) %>% add_candidates(log_res)
st_blend <- blend_predictions(st)
st_fit <- fit_members(st_blend)

st_fit
```

Output & Results

`fit_members()` returns the trained stack with meta-learner coefficients weighting each base model. Inspecting `st_blend` shows the chosen blending weights — typically a non-negative least-squares or penalised regression solution that emphasises the base models with the most distinctive predictive signal.

Interpretation

A reporting sentence: “Stacking a random forest and a regularised logistic regression via a penalised linear meta-learner achieved test ROC AUC of 0.89, exceeding either base model alone (RF 0.85, logistic 0.80); the meta-learner weighted RF at 0.62 and logistic at 0.38, with both contributions retained at non-zero values.” Always report base-model performances, the meta-learner choice, and the blending weights.

Practical Tips

- Base-model diversity (tree + linear + kNN + neural net) matters more than having many similar learners; a stack of five XGBoost variants is rarely better than a single well-tuned XGBoost.
- Use a regularised linear meta-learner (lasso or non-negative LS); unregularised meta-learners over-fit the relatively small set of OOF predictions.
- Pass *out-of-fold* predictions to the meta-learner, not in-sample predictions; this is the central correctness guarantee of stacking.
- Stacking plus bagging hybrids (super learner of van der Laan, Polley, Hubbard) perform best on moderate-to-large data.
- The `stacks` package (tidymodels) automates the workflow and integrates with the broader `tune` ecosystem for cross-validated base-model tuning.
- `SuperLearner` is the classical alternative outside tidymodels; both produce equivalent results given the same base models.

R Packages Used

`stacks` for tidymodels-based stacking; `SuperLearner` for the classical implementation; `tune::fit_resamples()` for the cross-validated base-learner predictions; `parsnip` for unified base-model specifications; `glmnet` and `ranger` for the standard base-learner choices.

For Reviewers

What to look for in a paper using this method.

- Common misapplications.
- Diagnostics that should be reported but often aren’t.
- Red flags in tables and figures.
- What to verify.
- What an adequate Methods paragraph must contain.

See also — labs in this chapter

- Cross-validation, nested CV, bootstrap .632+
- Regularisation: ridge, lasso, elastic net
- Trees, random forests, gradient boosting
- Interpretability and SHAP
- Tabular neural networks with torch
- Imaging and sequence models intro
- tidymodels pipelines
- FDR, knockoffs, replication crisis